

Approach & Key Decisions

Data Q&A Agent — Forward Deployed Engineer take-home

The approach

The obvious way to build this is to paste a spreadsheet and a question straight into an LLM prompt and let it answer. I didn't, because it doesn't really work: models are unreliable at arithmetic, they struggle once a file has more than a few hundred rows, and they have no principled way to join two files together. So instead of asking the model to be the calculator, I built it to be a translator — it turns a question into a SQL query, and DuckDB actually runs that query against the real data. Every number on screen was computed by a database engine, never guessed by a language model.

Key technical choices

DuckDB for the analytical engine:

- Embedded, so there's no separate database server to install or manage
- Built for ad-hoc analytical queries, not general-purpose transactional workloads
- Speaks real SQL directly against pandas DataFrames, so a multi-file join is a native JOIN, not custom code

Plotly for charts, not pandas' own .plot():

- Real interactivity (hover, zoom) inside Streamlit; pandas' built-in plotting only produces a static image
- Multi-series grouping (one line per region) through a single `color=` parameter, not hand-rolled per chart type

Skipping automatic join-key detection across files — it's a real time sink with realistic failure modes (EmpID vs. Employee_Code is exactly the kind of mismatch a scoring heuristic would miss), and the model writes correct joins on its own once given every table's columns and a few sample rows.

Streamlit over a custom React frontend — it covers everything the brief asks for, without a heavier local toolchain.

Keeping it safe

- Every generated query is parsed into a real SQL syntax tree and checked against an allow-list of uploaded tables, not just checked for the word SELECT — DuckDB treats `read_csv_auto('/etc/passwd')` as a perfectly legal SELECT.
- A failed query's exact error goes back to the model, with up to two chances to fix it.
- Chart type is chosen by deterministic rules based on the result's shape, not by the LLM — falling back to a table with a one-line reason whenever that shape is ambiguous.

About the model, honestly

The brief asks for an open-source model as the core engine, and I started with Groq for exactly that reason. Its free tier's rate limits got in the way of iterating quickly, so what's actually running is OpenAI's API — a real deviation, worth saying plainly rather than leaving it to be discovered. The client is generic enough that switching back to an open-weight model is a config change, not a rewrite.

The hardest bug

A question like "what does this file tell me about?" is completely reasonable, but no SQL query can answer it — there's no SELECT statement whose rows mean "this file tracks employee attendance." My first fix was a keyword shortcut to catch these questions and answer from the file's structure directly, but real phrasing broke it almost immediately. What held up was giving up on classifying the question at all, and letting the model choose, in the same response, between writing SQL or a plain description — which is what it's good at in the first place.

What I'd build next

- Give DuckDB somewhere permanent to live — it currently runs in memory inside the app's own process, so nothing survives a restart or is shared across instances.
- Add basic access control on the hosted link — right now anyone with the URL can spend against my OpenAI key, with no login or per-user limits.
- A small, fixed set of test questions with known-good answers, to catch regressions when the model changes.
- A safe way to handle genuinely statistical questions — forecasting, correlation — that don't fit a single SQL query.